UNIVERSITY OF KARACHI

Department of Computer Science (UBIT)

ASSIGNMENT NO. 02

Operating Systems (OS)

SUBMITTED BY:	SUBMITTED TO:
Shaikh Mahad UdDin Seat No: EB25210106113 Section: BSSE A Evening	**Ms.Nazish Imran** Faculty of Computer Science

---

Date of Submission: Aug 11, 2026

Assignment Overview

In this assignment, I have calculated the execution sequence and performance metrics for six different CPU scheduling algorithms based on the provided process table. For each algorithm, I have drawn the corresponding Gantt chart and calculated the Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), and Response Time (RT). Finally, I included a comparison to evaluate which algorithm performs best for this specific set of data.

Provided Process Data:

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P1	0	8	2
P2	1	4	1
P3	2	9	4
P4	3	5	3
P5	4	2	5

Priority Assumption: Following standard OS principles, I am assuming that a lower numerical value indicates a higher priority. Therefore, Priority 1 is the highest, and Priority 5 is the lowest.

Formulas used for the calculations:

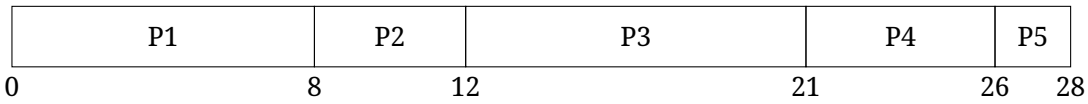
- **Turnaround Time (TAT)** = Completion Time (CT) - Arrival Time (AT)
- **Waiting Time (WT)** = Turnaround Time (TAT) - Burst Time (BT)
- **Response Time (RT)** = First CPU Start Time - Arrival Time (AT)

1. First Come First Serve (FCFS)

For FCFS, processes are scheduled purely in the order they arrive in the ready queue. Since it is non-preemptive, once a process gets the CPU, it runs until it is completely finished.

Execution Logic:
At $t = 0$, only P1 is available, so it executes first. After P1 finishes at $t = 8$, the remaining processes are executed in their exact arrival order: P2 $\rightarrow$ P3 $\rightarrow$ P4 $\rightarrow$ P5.

Gantt Chart:



Calculations:

Process	AT	BT	CT	TAT	WT	RT
P1	0	8	8	$8 - 0 = 8$	$8 - 8 = 0$	$0 - 0 = 0$
P2	1	4	12	$12 - 1 = 11$	$11 - 4 = 7$	$8 - 1 = 7$
P3	2	9	21	$21 - 2 = 19$	$19 - 9 = 10$	$12 - 2 = 10$
P4	3	5	26	$26 - 3 = 23$	$23 - 5 = 18$	$21 - 3 = 18$
P5	4	2	28	$28 - 4 = 24$	$24 - 2 = 22$	$26 - 4 = 22$

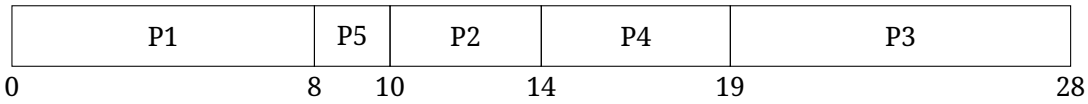
Average TAT = 17.0 ms | **Average WT** = 11.4 ms | **Average RT** = 11.4 ms

2. Shortest Job First (SJF - Non-Preemptive)

This algorithm selects the process in the ready queue that has the shortest burst time. Because it is non-preemptive, an executing process is not interrupted even if a new process arrives with a shorter burst time.

- At $t = 0$, P1 is the only process, so it starts.
- At $t = 8$, P1 finishes. The ready queue now has P2, P3, P4, and P5. I selected P5 next because it has the shortest burst time (2 ms).
- After P5, the remaining processes are selected in ascending order of their burst times: P2 (4 ms), P4 (5 ms), and finally P3 (9 ms).

Gantt Chart:



Calculations:

Process	AT	BT	CT	TAT	WT	RT
P1	0	8	8	$8 - 0 = 8$	$8 - 8 = 0$	$0 - 0 = 0$
P2	1	4	14	$14 - 1 = 13$	$13 - 4 = 9$	$10 - 1 = 9$
P3	2	9	28	$28 - 2 = 26$	$26 - 9 = 17$	$19 - 2 = 17$
P4	3	5	19	$19 - 3 = 16$	$16 - 5 = 11$	$14 - 3 = 11$
P5	4	2	10	$10 - 4 = 6$	$6 - 2 = 4$	$8 - 4 = 4$

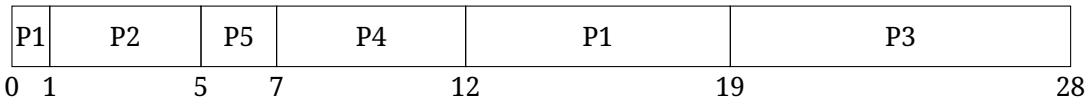
Average TAT = 13.8 ms | **Average WT** = 8.2 ms | **Average RT** = 8.2 ms

3. Shortest Remaining Time First (SRTF)

SRTF is the preemptive version of SJF. Every time a new process arrives, the scheduler checks if the new process has a shorter remaining burst time than the currently running process. If it does, the current process is preempted.

- At $t = 0$, P1 starts.
- At $t = 1$, P2 arrives with a burst time of 4 ms. P1 has 7 ms remaining. Since $4 < 7$, P2 preempts P1.
- At $t = 5$, P2 finishes. The available processes are P1 (7 ms left), P3 (9 ms), P4 (5 ms), and P5 (2 ms). P5 is selected because it is the shortest.
- At $t = 7$, P5 finishes. P4 has the shortest remaining time (5 ms), so P4 executes next.
- At $t = 12$, P4 finishes. P1 has 7 ms remaining while P3 has 9 ms remaining, so P1 executes next.
- At $t = 19$, P1 finishes and P3 executes until completion at $t = 28$.

Gantt Chart:



Calculations:

Process	AT	BT	CT	TAT	WT	RT
P1	0	8	19	$19 - 0 = 19$	$19 - 8 = 11$	$0 - 0 = 0$
P2	1	4	5	$5 - 1 = 4$	$4 - 4 = 0$	$1 - 1 = 0$
P3	2	9	28	$28 - 2 = 26$	$26 - 9 = 17$	$19 - 2 = 17$
P4	3	5	12	$12 - 3 = 9$	$9 - 5 = 4$	$7 - 3 = 4$
P5	4	2	7	$7 - 4 = 3$	$3 - 2 = 1$	$5 - 4 = 1$

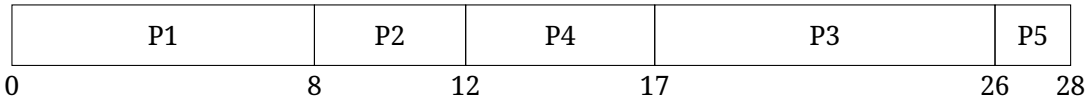
Average TAT = 12.2 ms | **Average WT** = 6.6 ms | **Average RT** = 4.4 ms

4. Priority Scheduling (Non-Preemptive)

This algorithm assigns the CPU to the process with the highest priority (which is the lowest number based on our assumption). Being non-preemptive, an active process won't be interrupted even if a higher priority process enters the queue.

- At $t = 0$, only P1 is available, so P1 executes first.
- At $t = 8$, P1 finishes. The available processes and their priorities are: P2 (1), P3 (4), P4 (3), and P5 (5).
- Since a lower number indicates a higher priority, P2 executes next.
- The remaining processes are then selected according to their priority: P4 (3) $\rightarrow$ P3 (4) $\rightarrow$ P5 (5).

Gantt Chart:



Calculations:

Process	AT	BT	CT	TAT	WT	RT
P1	0	8	8	$8 - 0 = 8$	$8 - 8 = 0$	$0 - 0 = 0$
P2	1	4	12	$12 - 1 = 11$	$11 - 4 = 7$	$8 - 1 = 7$
P3	2	9	26	$26 - 2 = 24$	$24 - 9 = 15$	$17 - 2 = 15$
P4	3	5	17	$17 - 3 = 14$	$14 - 5 = 9$	$12 - 3 = 9$
P5	4	2	28	$28 - 4 = 24$	$24 - 2 = 22$	$26 - 4 = 22$

Average TAT = 16.2 ms | **Average WT** = 10.6 ms | **Average RT** = 10.6 ms

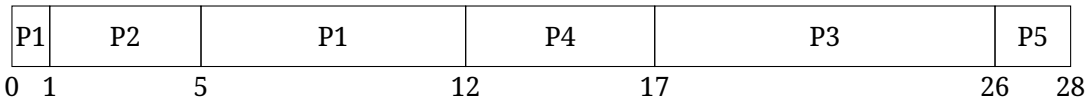
5. Priority Scheduling (Preemptive)

In preemptive priority scheduling, the CPU is allocated to the process with the highest priority (lowest numerical value). If a new process arrives with a higher priority than the currently running process, the CPU is preempted and assigned to the new process.

Execution Logic:

- At $t = 0$, P1 starts.
- At $t = 1$, P2 arrives with priority 1. P1 has priority 2. Since $1 < 2$, P2 preempts P1.
- At $t = 5$, P2 finishes. P1 has 7 ms remaining and has priority 2. Since P3, P4, and P5 have lower priorities, P1 resumes.
- At $t = 12$, P1 finishes. P4 is next because it has the highest priority (3) among the remaining processes.
- At $t = 17$, P4 finishes. P3 executes next (priority 4), followed by P5 (priority 5) at $t = 26$.

Gantt Chart:



Calculations:

Process	AT	BT	CT	TAT	WT	RT
P1	0	8	12	$12 - 0 = 12$	$12 - 8 = 4$	$0 - 0 = 0$
P2	1	4	5	$5 - 1 = 4$	$4 - 4 = 0$	$1 - 1 = 0$
P3	2	9	26	$26 - 2 = 24$	$24 - 9 = 15$	$17 - 2 = 15$
P4	3	5	17	$17 - 3 = 14$	$14 - 5 = 9$	$12 - 3 = 9$
P5	4	2	28	$28 - 4 = 24$	$24 - 2 = 22$	$26 - 4 = 22$

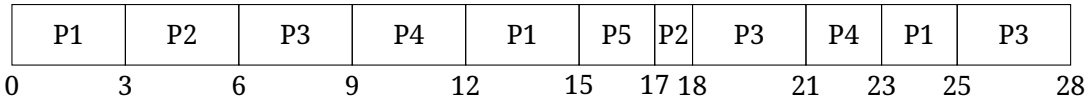
Average TAT = 15.6 ms | **Average WT** = 10.0 ms | **Average RT** = 9.2 ms

6. Round Robin (RR) - Time Quantum = 3ms

Round Robin gives every process a fixed time slice (quantum). Here, the quantum is 3 ms. If a process doesn't finish within its 3 ms window, it gets preempted and pushed to the back of the ready queue.

Ready Queue Execution Logic:
Time Quantum = 3 ms. At $t = 0$, P1 starts. During its execution, P2 and P3 arrive, and P4 arrives at $t = 3$. Therefore, after P1's first quantum expires, the ready queue is P2 $\rightarrow$ P3 $\rightarrow$ P4 $\rightarrow$ P1. The scheduler continues giving each process up to 3 ms in cyclic order. P5 joins the queue at $t = 4$. When a process requires less than 3 ms (like P5 or the final portions of P2, P4, and P1), it runs only for its remaining time and then completes, allowing the next process in the ready queue to execute.

Gantt Chart:



Calculations:

Process	AT	BT	CT	TAT	WT	RT
P1	0	8	25	$25 - 0 = 25$	$25 - 8 = 17$	$0 - 0 = 0$
P2	1	4	18	$18 - 1 = 17$	$17 - 4 = 13$	$3 - 1 = 2$
P3	2	9	28	$28 - 2 = 26$	$26 - 9 = 17$	$6 - 2 = 4$
P4	3	5	23	$23 - 3 = 20$	$20 - 5 = 15$	$9 - 3 = 6$
P5	4	2	17	$17 - 4 = 13$	$13 - 2 = 11$	$15 - 4 = 11$

Average TAT = 20.2 ms | **Average WT** = 14.6 ms | **Average RT** = 4.6 ms

7. Algorithm Comparison and Conclusion

The table below summarizes the calculated averages for each of the six scheduling algorithms so they can be easily compared.

Scheduling Algorithm	Avg WT (ms)	Avg TAT (ms)	Avg RT (ms)
First Come First Serve (FCFS)	11.4	17.0	11.4
Shortest Job First (SJF)	8.2	13.8	8.2
Shortest Remaining Time First (SRTF)	**6.6**	**12.2**	**4.4**
Priority Scheduling (Non-Preemptive)	10.6	16.2	10.6
Priority Scheduling (Preemptive)	10.0	15.6	9.2
Round Robin (RR - Quantum 3)	14.6	20.2	4.6

Final Analysis:

When deciding which algorithm is the best, it really depends on what performance metrics we want to prioritize for the system.

For this specific set of processes, **Shortest Remaining Time First (SRTF)** gives the best overall results if the main goal is to minimize both the average waiting time (6.6 ms) and average turnaround time (12.2 ms). Because it is preemptive, it stops short jobs from getting stuck behind longer ones. It also resulted in the lowest average response time (4.4 ms). However, it is important to note that SRTF has a known fairness issue: longer processes can suffer from starvation (for example, P3 had a long individual wait time of 17 ms).

On the other hand, **Round Robin (RR)** resulted in the highest average waiting time (14.6 ms) and turnaround time (20.2 ms). But despite those high averages, Round Robin generally provides better fairness and strong CPU responsiveness because each process receives regular CPU time. This is reflected in its very low average response time of 4.6 ms.

Conclusion: For the given process set, **SRTF** performs best when average waiting time, turnaround time, and response time are prioritized, as it produces the lowest values among the algorithms tested. However, **Round Robin** provides fairer CPU sharing and is more suitable for interactive systems where responsiveness and fairness are important. Therefore, **SRTF** is the best choice for this particular dataset based on the calculated performance measures.